

Documented, accepted, honoured: an audit of decoding parameters on hosted LLM inference APIs

Anonymous authors

Paper under double-blind review

Abstract

Evaluation results for hosted language models are reported with a decoding configuration — a temperature, a nucleus threshold, sometimes a seed — on the assumption that the provider applies what the request asks for. We test that assumption. For every chat model one provider serves (18 models, \$0.20 to \$15.00 per million output tokens) and every sampling parameter that provider documents (nine), we separate three facts that are routinely conflated: what the vendor’s reference *documents*, what the API *accepts*, and what the output distribution shows was *honoured*, the last established by a permutation test on 40 completions per arm, stratified across prompts, with a power-analysed positive control and a negative control calibrated on real output. Of 182 (model, parameter) cells, 98 behave as documented and 27 are documented, accepted and inert. The largest single gap is *seed*, described in the reference as “Random seed for deterministic sampling” and without measurable effect on 17 of 18 models; *mirostat* is accepted everywhere and inert on six models, which is the first parameter we find whose honouring differs *between models behind one serving stack*. The provider also accepts four parameters it does not document: two are inert, one — *ignore_eos* — works, pushing generation past the end-of-turn token and returning the raw control token in the response. Separately, only one model in 18 reproduces byte-identical output across ten identical, concurrently issued temperature-0 requests, and only two when the requests are issued one at a time: for eight models the probe’s own concurrency measurably lowers agreement, for the rest it does not. We release the instrument, the raw completions for every verdict, and the dated provenance for each. Because provider behaviour changes — three models were withdrawn during this study — every result is dated from the timestamp of the calls it was made of.

1 Introduction

A reported benchmark number is a function of the model, the prompt, the benchmark, and the decoding configuration. The first three are usually reported. The fourth is reported inconsistently, and when it is reported, it is reported as the *request* the harness sent, not as the behaviour the provider applied to it.

Those differ. This paper measures how much, on one hosted provider, across every chat model it serves.

Our contribution is an audit and an instrument, not a claim about how much decoding configuration moves benchmark accuracy — a question already addressed for self-hosted inference by Hochlehnert et al. (2025) and Pape et al. (2026), and which we discuss as future work in Section 7. The audit answers a prior question that those studies assume away by controlling the serving stack: on a hosted API, does the decoding parameter you sent do anything at all?

We make three distinctions the literature and the tooling collapse:

- **Documented** — the vendor’s API reference describes the parameter. We transcribe each provider’s reference verbatim, with URL and date of reading, and record “absent” as a positive statement about a page read on a date rather than as a missing key.
- **Accepted** — the HTTP status when the parameter is sent. A strict validator returning 400 `Extra inputs are not permitted` is a different fact from a 200.
- **Honoured** — the output distribution differs between two settings of the parameter, far apart in its documented range, with everything else held fixed.

The gap between the first and the third is the audit’s subject. Its worst case — accepted by the API, absent from the documentation, and inert at runtime — is a parameter a user can set and receive no error, no warning, and no effect from. We find nine such cells and name them.

2 Related work

Is the API serving the model it claims? Gao et al. (2025) test whether a hosted endpoint serves the advertised weights, using a maximum mean discrepancy two-sample test on completions across 31 endpoints and 9 providers. We share the statistical family — a distribution-level two-sample test on completions, since neither question can be settled from a single response — and ask a different question: not *which model* is behind the endpoint, but *what the serving stack does with the request’s decoding parameters*. Model equality testing holds the decoding configuration fixed as a nuisance; here it is the treatment. Karvonen et al. (2025) audit providers for quantization through a seeded verifier, again taking the decoding path as given. Li et al. (2026) measure hosted open-weight APIs at the service level — latency, throughput, price, protocol compatibility — and recommend, without running, exactly the decoding-compliance checks we implement.

Does decoding configuration move benchmark scores? Hochlehnert et al. (2025) show on local vLLM that seed alone moves AIME24 accuracy by 5–15 points and that temperature and `top_p` move it comparably; Pape et al. (2026) show that swapping inference backends moves GSM8K by up to 16.6 points, and trace part of it to a hidden default repetition penalty in one backend. Both establish that decoding configuration and serving stack matter for reported numbers, and both deliberately work where the stack is under the experimenter’s control. Our finding that a documented parameter can be accepted and inert is the hosted counterpart of their hidden-default finding: in one case the stack applies something you did not ask for, in the other it does not apply something you did.

Is temperature-0 decoding reproducible? Atil et al. (2024) report that identical requests at temperature 0 with a fixed seed agree at rates from 0% to 86% across five models. We replicate this on 18 hosted models behind one stack, and add the comparison the provider’s documentation invites: the reference promises “deterministic sampling” from `seed`, and we measure whether a fixed seed makes a non-deterministic prompt reproducible. He & Thinking Machines Lab (2025) identify batch-invariance as the mechanism, which makes a probe’s own concurrency part of the treatment; we measure it directly (Section 4.3).

Inferring the decoding strategy. Ippolito et al. (2023) recover k or p from unique-count statistics on engineered small-support prompts, given blackbox access. That is a strictly stronger measurement than ours, which is binary per cell (distinguishable/not), and it is the natural next version of this instrument: estimating the *effective* k a provider applies rather than only whether k is applied. It answers a different question, though — what the system does — rather than whether the system complies with what the caller asked for.

To our knowledge, no prior work tests per-(provider, model, parameter) compliance with user-supplied decoding parameters, compares it against the vendor’s own documentation, or reports parameters a provider accepts without documenting.

3 Method

3.1 Design

For each (model, parameter) we run a two-sample test: $n = 40$ completions at a *tight* setting and 40 at an *open* setting, far apart in the parameter’s documented range, with every other field of the request identical. Arms are named by direction rather than by label, so a truncation parameter’s tight arm is the one that should narrow the output distribution if honoured.

Statistic and null. The null is exchangeability of the two arms, which is exactly the hypothesis “the parameter did nothing”. We permute arm labels 10,000 times. For a truncation parameter (`top_p`, `top_k`, `min_p`, `typical_p`, `mirostat`) the primary statistic is the one-sided entropy drop $\Delta H = H(\text{open}) - H(\text{tight})$, since an honoured truncation reduces entropy in a known direction. For a penalty (`repetition_penalty`, `frequency_penalty`, `presence_penalty`) the direction is not known — a strong penalty can raise or lower the entropy of a short reply — so the primary is the two-sided total variation distance. This mattered empirically: on one model `frequency_penalty` emptied the open arm, giving $\Delta H = -3.3$ with one-sided $p = 1.0$ while the two-sided test saw the same shift at $p = 0.0002$. Holm–Bonferroni is applied across the grid; per-cell claims use the per-cell p -value and we report both.

Positive control. A null result is uninterpretable unless the probe is known to have power on that (model, prompt). The tight settings under test (`top_k=1`, `top_p=0.01`, `min_p=0.9`, `typical_p=0.1`) are greedy-equivalent when honoured, and every open arm is unrestricted sampling at temperature 1.0, so the largest effect an honoured parameter can produce is exactly the contrast between temperature 0 and temperature 1.0. Both arms are already collected. We estimate the test’s *power* to detect that contrast by bootstrapping the two arms and re-running the same test at the same n , at the Holm-corrected level the parameter tests face (α/m , with m the tests in the family), and call a prompt powered when that power is at least 0.8. A null on an unpowered prompt is reported as *underpowered* — evidence of nothing — rather than *no effect seen*. “No effect seen” therefore means that a fully honoured setting would have been detected with at least 80% power and was not; it does not exclude a partially honoured parameter with a smaller effect.

Two earlier versions of this control are worth recording because each failed in a way that changes verdicts. The first contrasted temperature 0 with 1.5 and required half the open-arm entropy to be removed: the range was wrong (the arms under test sit at 1.0, and at 1.5 several reasoning models run to the token cap and return nothing, collapsing the open arm), and the threshold had no derivation. The second required only that the correct contrast be *detectable* at $p < 0.05$. That is too weak: a control that clears the threshold by a hair has roughly even power, and the parameter tests face a multiplicity correction on top; it turned several truncation cells on low-headroom models from undetermined into “inert”. The power criterion reverses those. The three-way distinction between “the provider refused it”, “the provider took it and nothing changed”, and “the probe could not have seen a change” is the reason the instrument exists; an earlier boolean version of this classification conflated the first two and nearly excluded a model on a $p = 0.063$ cell.

For the penalties the temperature control is the wrong control, because a penalty acts only on tokens that have already appeared and a one-sentence reply has few. Those parameters are additionally tested on a prompt that *forces* repetition (“Repeat the word ‘yes’ twenty times”), where the penalty must act if it is applied. That cell is its own control, and we report “applied” (forced prompt) separately from “matters on free text”.

Negative control. Two arms at *identical* settings must not be distinguishable. Collected in the same order as a real test, so that anything in the collection process that breaks exchangeability — provider state drifting between the first arm and the second — is exposed rather than assumed away. Pairs where both arms are all-unique are excluded: the statistic has no range there and $p = 1$ by construction, so counting them flatters the calibration. Pooled over the informative pairs, the primary statistic’s false-positive rate is 0/37.

Prompts and aggregation. Four prompts with genuine output entropy, each reported on its own. The aggregate verdict for a (model, parameter) is a stratified permutation test: arm labels are permuted *within* each prompt, the per-prompt statistic is summed, and the observed sum is compared with that null, giving one exact p -value however many prompts contribute. Holm–Bonferroni is applied across the (model, parameter) family. Because the sum tests the effect on average, an effect confined to one prompt can wash out, so if any single prompt shows the effect at its own corrected level the aggregate is reported as *mixed*, never as inert. Prompts on which both arms are all-unique or both constant are excluded: the statistic has no range there.

An earlier version aggregated by majority vote over prompts, which has no stated error rate. Kept for comparison, it agrees with the stratified test on 111 of 187 aggregates. Of the 76 disagreements, 46 are cells the vote left undetermined that pooling resolves; 20 become *mixed*; 4 move from mixed to distinguishable; and 6 are reversals of a definite verdict — all six from the vote’s “no effect seen” to an effect the pooled test detects, none in the other direction. One of those six is a model the first version of this paper listed as ignoring mirostat.

3.2 Determinism

Ten byte-identical requests at temperature 0, under three conditions (no seed, `seed=0`, `seed=1`), on five prompts per model — the four above plus one real MATH-500 problem, so the measurement speaks to the greedy cell an evaluation would actually run. The ten requests are issued five at a time; a separate same-day arm issues them strictly one at a time (Section 4.3). We report the modal share, and for the concurrency comparison also pairwise agreement (the share of the 45 pairs that are byte-identical), since the two are easily read for each other: the fraction of the ten responses equal to the most common response. The test for “is the seed honoured” is whether ten calls with the *same* seed agree, not whether `seed=0` and `seed=1` differ; on a model whose seeded runs are 40% self-consistent the latter differ because everything differs.

3.3 Provenance

Provider behaviour changes: three models were withdrawn from the catalogue during this study, and one returned. Every response is cached under a hash of its exact request body plus a replicate index, with the time the provider answered; every reported cell carries the collection window derived from those timestamps, not from when a report was written. Documentation is transcribed with its URL and date of reading. The raw completions for every verdict are released, so every verdict re-derives without the cache and without a key.

4 Results

4.1 Documented versus honoured

Table 1 summarises 182 cells: 18 models $\times$ nine documented parameters, plus four undocumented parameters on a five-model subset.

The largest documented gap is `seed`. The reference entry reads, in full, “Random seed for deterministic sampling.” Across 73 (model, prompt) cells that were not already deterministic without a seed, a fixed seed made the run reproducible in 10, and on no model in more than half of its non-deterministic prompts. Seventeen of eighteen models are therefore *documented*, *accepted*, *ignored* for this parameter. A harness that writes `seed=0` into a configuration and “reproducible” into a paper is relying on this sentence.

Honouring is not uniform behind one serving stack. For the truncation parameters no model is inert: every cell is either honoured or undetermined, which suggests that compliance is a property of the stack rather than the model. Mirostat refutes that: accepted on all 18, honoured on six, and inert — with at least 80% power to have seen a full-strength effect — on six others (deepseek-v4-flash-0731, deepseek-v4-pro-0813, glm-5p2, muse-glimmer-30b, qwen3p8-max, qwen3p8-2p4t-a95b). `typical_p` is inert on one model, which also ignores mirostat and `repetition_penalty` — three accepted, inert parameters on a single model behind a stack that honours them elsewhere.

Table 1: Every sampling parameter the provider documents, across all 18 chat models it serves. *Accepted* counts HTTP 200; *honoured* counts cells distinguishable by the stratified test after Holm correction; *inert* counts nulls with at least 80% power to have seen a fully honoured setting; *undet.* counts cells without that power, cells where every prompt was degenerate, and cells where prompts disagree (*mixed*). Collected 2026-09-17 to 2026-09-22.

Parameter	Documented as	Accepted	Honoured	Inert	Undet.
<code>top_p</code>	nucleus sampling	18/18	16	0	2
<code>top_k</code>	top- <i>k</i> filtering	18/18	16	0	2
<code>min_p</code>	min. probability threshold	18/18	15	0	3
<code>typical_p</code>	typical- <i>p</i> sampling	18/18	13	1	4
<code>mirostat</code>	target perplexity	18/18	6	6	6
<code>repetition_penalty</code>	penalty above 1.0	18/18	14	1	3
<code>frequency_penalty</code>	frequency penalty	18/18	18	0	0
<code>presence_penalty</code>	presence penalty	18/18	0	2	16
<code>seed</code>	“deterministic sampling”	18/18	0	17	1

Table 2: Parameters the provider accepts and does not document. Each is tested by a design in which an applied parameter must show: beam search must make temperature-1.0 sampling deterministic; `skip_special_tokens`=false must expose an end-of-turn marker; `ignore_eos` must push a one-word answer to the token cap. Collected 2026-09-22.

Parameter	Documented	Accepted	Honoured
<code>use_beam_search</code>	absent	5/5	inert on 4 , undet. on 1
<code>skip_special_tokens</code>	absent	5/5	inert on 5
<code>ignore_eos</code>	absent	5/5	honoured on 3 ; HTTP 500 on 1
<code>best_of</code>	absent	5/5	no signature observable externally

Applied is not the same as consequential. `frequency_penalty` at its maximum breaks forced repetition on 18 of 18 models, so it is applied everywhere. On free-text prompts it moves the output distribution on only 8 of 18; on 6 it is inert with power, and on 4 the prompts disagree. Both are true, and a harness reporting “`frequency_penalty=2.0`” is reporting a setting that is real and, for short generation on most of these models, inconsequential. `presence_penalty` is undetermined by this design: bounded at 2 logits, it cannot overturn a forced token, so the forced prompt cannot separate “not applied” from “applied but too small to matter”. We report it as undetermined rather than resolving it by assumption.

4.2 Accepted, undocumented, inert

The provider validates request bodies strictly: of fifteen sampler names drawn from other inference stacks (`top_a`, `tfs`, `dry_multiplier`, ...) all fifteen return 400 **Extra inputs are not permitted**. Four names from vLLM’s `SamplingParams` pass validation and appear nowhere in the reference. Table 2 gives the result of a decisive test for each on five models.

A caller who sets `use_beam_search=true` receives HTTP 200, no warning, and sampling. These nine cells are the audit’s worst case. Its mirror is arguably worse for evaluation: `ignore_eos` is *undocumented and honoured*, pushing the model past its end-of-turn token, with the raw control token appearing in the returned text (`cat<|assistant|>We need answer...`). Nothing in the reference says it exists.

We also record that `repetition_penalty=2.0`, the documented maximum, is accepted and then hangs: generation streams roughly 300 chunks in 7.6 seconds and then stalls until the connection drops at 742 seconds, with one non-streaming call held for 55 minutes. “Accepted and unusable at a documented value” is a fourth category, distinct from inert.

4.3 Temperature-0 decoding is not reproducible

One model in 18 returns byte-identical output across ten identical temperature-0 requests issued concurrently. The rest range from 20% to 78% modal agreement (the share of the ten equal to the most common response), and price does not predict it: a \$15.00/M model reproduces at 74%, a \$0.50/M model at 76%.

Batch composition is a known mechanism (He & Thinking Machines Lab, 2025), so concurrency is part of the treatment. On 2026-09-23 we ran both arms on the same day for all 18 models — ten requests issued one at a time, and the same ten in flight together — alternating which arm went first by prompt so a time trend could not load onto one arm. Two models reproduce on every prompt when sequential, against one when concurrent. Across the 90 (model, prompt) pairs, pairwise agreement is higher sequentially in 48, lower in 25, equal in 17; mean modal share is 70% sequential against 61% concurrent. The effect is concentrated: for eight models pairwise agreement rises by at least ten points when sequential, and one — nemotron-3-ultra — is fully deterministic sequentially (100%) and far from it concurrently (56% modal, 39% pairwise). For others, including kimi-k2p6 at 22% in both arms, the nondeterminism is independent of this client’s load. “Sequential” means sequential from the client; the provider still batches our request with other traffic, which no client controls. On the embedded MATH-500 problem, six models reproduce 10/10 and the worst reproduces 2/10 — for those, a reported greedy accuracy is one draw from a distribution, not a fixed quantity.

4.4 Grader failure modes are temperature-correlated

Hand-verifying an exact-match grader with normalisation against 50 stratified samples moved it from 30/50 to 50/50 agreement, finding five distinct bugs. The one that matters methodologically: a last-number fallback credited truncated responses, and in one case the number the model was last manipulating *was* the gold answer, scoring a response that never stated an answer as correct. Several of these bugs track temperature, because high temperature produces longer and messier output; a grader whose error rate rises with temperature injects sampler-correlated error into exactly the quantity a decoding study reports. After the fixes the lenient path is used 0% of the time in every configuration.

The three-valued verdict (correct / incorrect / unparseable) is load-bearing. Unparseable is not uniform across models: on a 1000-generation corpus one model of four produces unparseable responses at 10% (greedy) rising monotonically to 18% (temperature 1.0), and 27–33% on an AIME subset, while the other three produce none. Nearly all are responses that exhausted the token budget mid-reasoning. Scoring them as incorrect gives that model alone a +0.06 temperature effect — which vanishes among the responses it finishes, where it is the most accurate model of the four — and inverts 12 of 30 pairwise model comparisons. Whether to fold unparseable into “incorrect” is a choice, it is model-specific, and it is temperature-correlated, so it must be reported rather than absorbed.

5 Why an ignored parameter corrupts a decoding study

The audit’s practical consequence is best shown on the analysis it threatens. Consider two grid cells: {temperature 1.0, min_p 0.05} and {temperature 1.0, top_p 1.0}. On a stack that applies min_p these are two conditions. On a stack that accepts and ignores it they are the same condition run twice. If a grid mixes the two kinds of model — which the results above show is possible within a single provider — the contrast between those cells is real for some models and zero for others, and a two-way decomposition books the difference as a model × sampler interaction.

We illustrate this on a small corpus (three models, ten MATH-500 problems, five replicates) using real generations for both cases: a genuine min_p cell, and additional replicates of the top_p cell, which is by definition what an ignoring stack returns for the min_p request. With every stack honouring the parameter the interaction share is 7.9%; with one model’s stack ignoring it, 3.3–5.6% depending on which model; with all ignoring, 1.0%; with all ignoring but one, 1.7–3.4% manufactured from that 1.0%. Each is a legitimate analysis of a legitimately collected accuracy table, and the table carries no trace of which case it is. Only a compliance probe distinguishes them.

We stress that these magnitudes are those of a ten-problem grid and are an illustration of the mechanism, not an estimate of its size in practice.

6 Limitations

One provider. All 18 models sit behind one serving stack. The mirostat result shows that honouring can differ between models within a stack, but the cross-provider question — whether the same model behind two providers complies differently — is the natural next measurement and is not answered here. The instrument is provider-agnostic: each provider sits behind an adapter and the probe is invoked with a provider name.

The power estimate is a bootstrap. Power is estimated by resampling the observed control arms, which treats the observed arms as the truth; at $n = 10$ (four of the 18 models) this is optimistic, and those rows are marked thin in the released tables. It is computed for the reference effect of a fully honoured setting, not for the parameter’s own observed effect, so it is not post-hoc power in the usual, circular sense.

Drift. Repeating cells a few days apart moved some quantities materially (one model’s distinct-count at temperature 0 went from 11/40 to 20/40 between 2026-09-17 and 2026-09-20). This is a finding about hosted providers as much as a limitation, and it is why every cell is dated.

7 The variance study is future work, and why

This instrument was built for a different study: a fully crossed model $\times$ sampler $\times$ problem $\times$ replicate design, reporting the share of benchmark-accuracy variance attributable to decoding configuration against the share attributable to model choice. We are not reporting that study, and the reason is not compute.

A variance component estimated from k factor levels has relative scatter of roughly $\sqrt{2/(k-1)}$: 100% at three levels, 82% at four, 58% at seven. Simulating the frozen grid, a level-aware interval for the sampler-to-model ratio clears a threshold of 0.1 only 57% of the time *when the two components are exactly equal*. A three-model grid can measure that ratio but cannot test it. The grid as designed costs roughly \$16 and was affordable; it was not run because its headline quantity would not have been testable. More problems, more replicates and more spend do not change this. More model levels do, and a near-peer band on a single provider does not contain enough of them — which is the same conclusion the audit reaches from the other direction, and the reason both point at cross-provider work.

A defensible version of the consequence study, which we leave to future work, reports fixed effects for the models actually run plus the comparison-inversion rate — how often a ranking between two models flips when only the sampler changes — with an interval bootstrapped over problems. That statistic counts outcomes over observed cells and needs no few-level extrapolation.

8 Reproducibility

The instrument, the raw completions behind every verdict, the documentation transcription, and the dated provenance are released. Every table in this paper regenerates from committed records with no API key and no network: the response cache is not required, because each probe cell retains its completions. Verdicts can therefore be re-derived, and re-derived under a *changed* test, which we exercised when the degeneracy rule and the penalty statistic changed after collection. The continuous integration configuration runs the test suite and regenerates every report offline on three Python versions.

References

Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of “deterministic” LLM settings. *arXiv preprint arXiv:2408.04667*, 2024.

- Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations (ICLR)*, 2025. [arXiv:2410.20247](#).
- Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference, 2025. Thinking Machines Lab, Connectionism.
- Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandaraao, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.
- Daphne Ippolito, Nicholas Carlini, Katherine Lee, Milad Nasr, and Yun William Yu. Reverse-engineering decoding strategies given blackbox access to a language generation system. *arXiv preprint arXiv:2309.04858*, 2023.
- Adam Karvonen, Daniel Reuter, Roy Rinberg, Luke Marks, Adrià Garriga-Alonso, and Keri Warr. DiFR: Inference verification despite nondeterminism. *arXiv preprint arXiv:2511.20621*, 2025.
- Haorui Li, Zhenghui He, Xuanzi Liu, Yang Xu, Dongsheng Liu, Jiakang Ma, Lupan Wu, Yangjie Wu, Xiongchao Tang, and Tianhui Shi. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs. *arXiv preprint arXiv:2605.02821*, 2026.
- David Pape, Jonathan Evertz, and Lea Schönherr. The silent hyperparameter: Quantifying the impact of inference backends on LLM reproducibility. *arXiv preprint arXiv:2605.19537*, 2026.